Janiya Green

Iyana Halliburton

Implementation

1. Widget tree & ownership

Show one reusable StatelessWidget and one StatefulWidget in your app. Explain their responsibilities, how build() renders current values, and which interaction requires mutable state.

- Reusable StatelessWidget: MissionMatchSuggestionCard
  - This widget displays the Mission Match recommendation. It receives a MissionMatchResult and displays the suggested challenge, category, score, explanation, and weekly progress. It also has two buttons: View mission and Not now.
  - It is a StatelessWidget because it does not manage its own changing data. The parent provides the recommendation and the button callbacks. Its build() method renders the current values it receives.
- StatefulWidget: HomeScreen
  - HomeScreen manages the list of challenges and their completion status. It uses setState() when a challenge is added or marked complete.
  - Its build() method reads the current challenge list, calculates how many challenges are completed, and updates the progress bar and challenge list on the screen.
- The interaction that requires mutable state is checking off a challenge or adding a new challenge. Those actions change the data, so the screen needs to rebuild to show the updated values.

2. Buttons, controls & state

Trace a button and slider interaction (or justify your numeric-control alternative) through input validation, setState()/state notification, and rendering. Which values are transient and which must be saved?

- Completing a challenge:

1. The user taps the checkbox
2. The button calls _doneChallenge(index)
3. The method uses setState()
4. The challenge's isCompleted value changes from false to true or vice versa
5. Flutter rebuilds the screen and all affected states update

- Temporary UI State values are transient.
- Challenge completion status is saved because it needs to remain updated

Architecture

1. Lifecycle

Show how a controller, timer, listener, or async operation is owned and cleaned up. What happens when a user leaves the screen before an operation finishes?

- When a user leaves the screen before an operation finishes, the asynchronous database operation can continue running, but the screen checks whether it is still mounted before updating its UI. This prevents the app from trying to call setState() or use the navigation context after the screen has been removed.

2. Testable boundaries

How do repository boundaries and dependency injection let you test without a real device database? Show the dependency direction and a fake or test implementation.

- Our app separates the interface from the database helper. The screens call DatabaseHelper methods instead of writing SQL directly. The database helper handles creating, reading, updating, and deleting records.
- Dependency Direction:

  Home Screen

  |

  V

  Challenge Detail Screen

  |

  V

  Database Helper

  |

  V

  SQLite Database

Local Data

1. Both entities' CRUD

Show create, read, update, and delete for your primary record and linked history record. Trace SQL/helper calls, validation, delete confirmation, and the final refreshed UI. Explain what happens to child records when a parent is deleted.
—------
Primary Record
Create: The user fills out the challenge form. The app validates the required fields, then calls the database helper to insert the challenge into SQLite.
Read: The app retrieves saved challenges from SQLite and displays them on the home screen.
Update: The user edits a challenge. The form validates the changes and calls the helper to update the database record.
Delete: The user taps delete and confirms through a confirmation dialog. The helper deletes the challenge, and the screen refreshes to show the updated list.

Linked History Record
Create: When the user completes an activity, the app creates a progress log linked to that challenge.
Read: The app retrieves progress logs and displays the user's activity history.
Update: Progress is updated when the user checks or unchecks an activity. The app updates the completion state and adds or removes the related progress log.
Delete: When an activity is unchecked, the related progress log is deleted from SQLite.

What happens to child records when a parent is deleted:

- Activities and notes are designed to be deleted automatically when their parent challenge is deleted (CASCADE). Progress logs are designed to prevent a challenge from being deleted while related logs still exist (RESTRICT).
- After database changes, the app refreshes the displayed data so the user sees the latest information.

2. Persistence & no-cloud proof

Explain SQLite versus SharedPreferences in your app. With networking disabled, save records and a setting, close/relaunch the app, and show restored values. Why are GitHub, ClickUp, and YouTube not app backends?

- SQLite: Stores challenges, activities, notes, and progress logs. This includes information such as challenge titles, categories, target amounts, completion status, and recorded progress.
- SharedPreferences: Stores the user’s preferred challenge category for Mission Match.

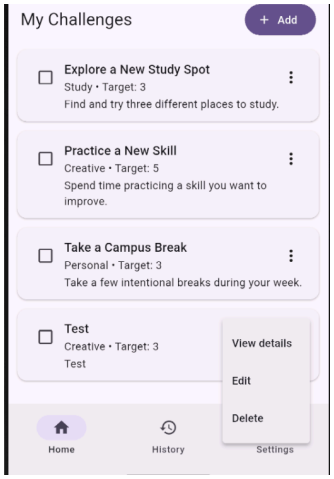
- GitHub, ClickUp, and YouTube aren't app backends because none of them are responsible for getting any app records.

Testing

1. Automated tests

Demonstrate unit, widget, and integration tests and explain their different boundaries.

- Unit tests check individual pieces of logic without testing the entire app. They only check individual logic.
- Widget tests check how individual screens and interface components behave. They only check interface behavior.
- Integration tests check whether multiple parts of the app work together. They check the complete workflow between screens and local storage.



2. Deterministic state transitions

How do you test start/progress/finish/reset and lifecycle cleanup without relying on arbitrary waiting? Show controlled inputs or time and prove reset does not leave stale callbacks active.

- I test it using controlled time instead of waiting for real seconds. I use a fake timer or controlled clock to test each stage:
  - Start: Start the timer and check that it begins at zero
  - Progress: Advance the fake clock by a set amount and verify the displayed time updates correctly.
  - Finish: Advance to the end time and check that the timer stops.
  - Reset: Reset the timer and confirm it returns to zero and can start again

- For lifecycle cleanup, I reset or leave the screen, advance the fake clock again, and verify that nothing updates. This proves old timer callbacks were canceled and are not still running in the background.

Reflection

1. Authentic ownership

Explain your contribution with a feature branch, meaningful commits, and a review. How did the team distribute work and ensure everyone could explain their implementation?

Janiya

- Feature Branch: `SQLite`
- Meaningful Commits: `add Databasehelper, fix app screens and improve implementation`
- Review: I was mainly focused on finishing the database implementation and making sure it actually worked with the code we already had. I had to connect the database methods to the screens so that challenges and their related information could be saved instead of only existing temporarily while the app was open.

Iyana

- Feature Branch: Challenge Screens
- Meaningful Commits: tested home screen with nav bug fixedAdding challenge form, Update Documentation Log
- Review: I focused on the front-end work, regarding the implementation and creation of the challenges, as well as the interface and navigation. I had to ensure that the navigation plan did not stop the user from being able to access the additional pages and features we added. I also ensured that all items in display were accessible, back-end validation existed, and items that can be toggled saved.

Work Distribution & Implementation:

- We divided the work based on the positions and responsibilities we originally chose. We collaborated back and forth to get the project completed and make sure the different parts worked together. When something was not working, we communicated and worked through the problems instead of treating each part as completely separate.

2. Required smart helper & critical thinking

Demonstrate your local helper's input, suggestion, explanation, override, and no-data/invalid feedback. Why did you choose these inputs and this method? Compare with your baseline, show a counterexample, and justify a revision or retained limitation. Undergraduates show five case types

- Our Mission Match feature is a local, rule-based helper that recommends a challenge based on the user's preferred category and their progress during the current week.

Inputs:

- Challenge ID
- CHallenge title
- Challenge Category
- Challenge target units
- Progress logs
- User’s saved preferred category

We chose these inputs because they provide enough information to make a useful recommendation without collecting unnecessary information or needing an internet connection.

Baseline:

Our baseline recommendation is the valid challenge with the lowest ID. It does not consider the user's preferred category or weekly progress. Mission Match improves on this by considering both the user's category preference and whether they have met their weekly target.

Counterexample:

Consider the following-

Challenge | ID | Category | Weekly Progress | Target

—---------

Study in the Library - 1 - Study - 5 - 5

Practice Drawing - 2 - Creative- 0 - 3

Join Campus Event - 3 - Social- 0 - 2

—---

If the user's preferred category is Creative, the baseline would recommend Study in the Library because it has the lowest ID. Mission Match would recommend Practice Drawing because it matches the preferred category (+2) and its target is unfinished (+1), giving it a score of 3.

Revision or Retained Limitation:

One limitation is that the helper only considers category preference and weekly progress. It does not consider deadlines, difficulty, available time, or how long a challenge might take. We kept the rules simple and predictable so users can understand why a challenge was recommended.

Case Types:

Case	Input	Expected Result
Category match	Preferred category matches an unfinished challenge	+3 points
No category match	Different category, target unfinished	+1 point
Target met	Matching category, target already met	+2 points
Empty catalog	No challenges exist	"Add a challenge first."
Invalid data	All challenges have invalid values	"No valid missions are available."

